CSCP2023: Data Structures & Algorithms (Spring 2025)

Assignment 4

(Deadline: 30th June, 2025 2:00 PM)

1. **Submission:** You are required to use Visual Studio for the assignment. Combine all your work (solution folder) in one .zip file after performing "Clean Solution". Name the .zip file as ROLL_NUM_04.zip (e.g. **0001_04.zip**). Submit zip file on slate within given deadline. Failure to submit according to above format would result in deduction of 10% marks.
2. **Comments:** Comment your code properly. **Bonus marks (maximum 10%)** will be awarded to well commented code.
3. **Deadline:** Deadline to submit assignment is **30th June, 2025 2:00 PM**. Correct and timely submission of assignment is responsibility of every student; hence no relaxation will be given to anyone.
4. **Plagiarism: -50% marks** in the assignment if any significant part of assignment is found plagiarized. A code is considered plagiarized if **more than 20%** code is not your own work.

Objective:

Implement a **Min Heap** to simulate a **task scheduler** that always picks the task with the **earliest deadline**.

📄 **Background Scenario:**

A software development company manages multiple tasks, each with an associated **task name** and a **deadline in hours** (e.g., 4 means due in 4 hours). The scheduler should always pick and execute the task that is **due the soonest**.

To efficiently manage this, you need to implement a **Min Heap** where the **minimum deadline task** is always at the top.

🔧 **Assignment Tasks:**

1. Design the Min Heap:

- Create a struct/class Task with:
  - string name
  - int deadline
- Implement the following heap operations:
  - void insert(Task task)
  - Task extractMin() // removes and returns the task with the earliest deadline
  - bool isEmpty()

- void displayHeap() // show all tasks in heap order (for debugging)

2. Simulation:

- Insert at least **10 different tasks** with random deadlines.
- After all insertions, simulate the execution of tasks by repeatedly calling extractMin() and printing them in order of execution.

Sample Input:

insert(Task("Code Review", 5));

insert(Task("Write Report", 2));

insert(Task("Client Meeting", 4));

insert(Task("Email Update", 1));

...

Expected Output:

Executing: Email Update (Deadline: 1)

Executing: Write Report (Deadline: 2)

Executing: Client Meeting (Deadline: 4)

Executing: Code Review (Deadline: 5)

...

3. BuildHeap

Accept task input from a file (e.g., tasks.csv with name and deadline columns).

This should first create a binary tree and then ensure the heap and structure property at the end.